ALLIANCE UNIVERSITY
Alliance School of Advanced Computing
Computer Science and Engineering · Kalvium

MINI PROJECT REPORT

DFA-BASED ASSEMBLY LEXER FOR A RISC PIPELINE SIMULATOR

Formal Language and Automata Theory
Concept: Deterministic Finite Automata and tokenization (Modules 1–2)

Integrated project with COA and DAA — one common application, three separate reports

Submitted by
Dasari Saideep Srinivas
University Enrolment No. 2411021060596
Squad 70

In partial fulfilment for the award of the degree of
BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
Vaibhav Giri
Technical Mentor, Kalvium × Alliance

October 2026

COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on Formal Language and Automata Theory is submitted by Dasari Saideep Srinivas [Uni. Enrolment No. 2411021060596] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

Vaibhav Giri
Technical Mentor
Kalvium × Alliance University

Date: _____ Signature: _____

COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled DFA-BASED ASSEMBLY LEXER FOR A RISC PIPELINE SIMULATOR, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Vaibhav Giri, Technical Mentor.

I further confirm that the views, findings and conclusions expressed here come from my own effort and from running the program I built. This project is an integrated work shared with Computer Organization and Architecture and Design and Analysis of Algorithms. Each subject has its own report explaining the concept it contributes. The FLAT report covers the DFA-based lexer component only.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Dasari Saideep Srinivas
2411021060596

Name of the Student University Registration Number Signature

_____ _____ _____

Acknowledgement

I would like to thank everyone who helped me finish this integrated microproject. The project is a RISC pipeline simulator that combines concepts from three subjects: Formal Language and Automata Theory, Computer Organization and Architecture, and Design and Analysis of Algorithms. This report covers the FLAT portion.

I thank my technical mentor, Vaibhav Giri, for the project guidelines and for making it clear that concepts from Modules 1–3 had to be visible in a working program. The suggestion to combine multiple subjects into one application pushed me to find a project where each subject has a genuine role instead of being tacked on.

The FLAT lectures on DFA, NFA, and regular languages gave me the base for the lexer. I did not use a parser generator. I wrote a transition table by hand, the same way we drew DFA diagrams on paper, except that the states are named START, IDENT, REG_DIG and so on, and the alphabet is character classes like alpha, digit, sign, hash.

Building a real tokenizer forced me to appreciate why DFAs work: one pass, one state at a time, no backtracking. That is the same guarantee we proved in Module 1 and the same idea that makes grep fast.

Thank you.

Dasari Saideep Srinivas

TABLE OF CONTENTS

Acknowledgement 4
Abstract 6
Chapter I – Introduction 7
  Problem Statement 7
  Theory 8
Chapter II – System Configuration 9
  Project Links 9
  Algorithm 9
  Source Code 10
Chapter III – Results 11
Chapter IV – Conclusion 13
References 13

Abstract

This report covers the Formal Language and Automata Theory component of an integrated microproject that builds a RISC pipeline simulator. The simulator has three pieces: a DFA-based assembly lexer (FLAT, Modules 1–2), a five-stage pipeline engine with hazard detection (COA, Module 3), and a greedy forwarding scheduler (DAA, Module 3). Each subject has its own report; this one deals with the lexer.

The lexer takes raw assembly text and produces a stream of typed tokens: opcodes, registers, immediates, labels, and punctuation. It does this by running each character through a deterministic finite automaton whose transition table is coded explicitly in JavaScript. The DFA has 14 states and recognises 12 character classes. After tokenisation, a recursive-descent parser builds structured instruction objects that the pipeline engine can execute.

The practical outcome is that malformed instructions are caught before they reach the pipeline, and every token on the screen is colour-coded by its DFA-determined type. The theoretical link is Module 1 (DFA definition and simulation) and Module 2 (regular expressions as patterns that a DFA can recognise). No regex library is used; the DFA transition table is the pattern description.

Chapter I – Introduction

In every compiler or interpreter, the first step is lexical analysis: turning a flat string of characters into a sequence of meaningful tokens. The tool that does this is a scanner, and at the heart of every scanner is a finite automaton. In FLAT we draw DFAs on paper and prove they accept the right language. In this project I had to make one work on actual assembly code.

The assembly language I chose is a simplified RISC instruction set. It has R-type instructions like ADD R1, R2, R3, immediate instructions like ADDI R1, R2, 10, memory instructions like LW R1, 0(R2), branches, jumps, labels, and comments. Each of those is a different token type, and the DFA has to tell them apart while reading characters one at a time, left to right, without backtracking.

I picked this problem because the lexer feeds directly into the COA pipeline simulator. If the tokens are wrong, the pipeline runs the wrong instruction. So correctness matters, and a DFA gives an easy proof of correctness: if the transition table is right, the tokens are right. That is the same reasoning we use in Module 1 when we argue that a DFA recognises exactly the strings in a regular language.

Problem Statement

Given a multiline string of RISC assembly, split it into a stream of tokens where each token has a type (OPCODE, REGISTER, IMMEDIATE, LABEL_DEF, LABEL_REF, LPAREN, RPAREN, COMMA) and a value. The splitting must be done by a deterministic finite automaton whose transition table is explicit and inspectable, not hidden inside a library regex. Malformed input must produce an ERROR token with a line number.

Theory

A deterministic finite automaton is a 5-tuple (Q, Σ, δ, q0, F). In this project Q has 14 states: START, IDENT, REG_R, REG_DIG, IMM_SIGN, IMM_DIG, HASH, LPAREN, RPAREN, COMMA, COLON, COMMENT, WS, and ERROR. The alphabet Σ is the set of character classes: alpha, digit, sign, hash, comma, lp, rp, colon, ws, nl, eof, other. The start state is START. The accepting states depend on context: when the DFA reaches a state that cannot continue (the next character's class leads to a different initial state), it emits a token whose type depends on the state it stopped in.

This is the same "longest match" rule used by real scanners (Aho et al., 2006). The DFA runs until no transition continues the current token, then it emits what it has and resets to START. Because every transition is deterministic, the scanner runs in O(n) where n is the length of the source, with exactly one state lookup per character.

After tokenisation, a parser groups tokens into instruction objects. The parser is recursive-descent, not a DFA, but it only works because the token stream is clean. If the DFA misclassifies a character, the parser sees the wrong type and rejects the line.

Chapter II – System Configuration

The entire project is a static web application. No compiler, no server, no build step. I developed and tested it on Windows 10 with Google Chrome. The language is JavaScript (ES6). To run the project, open pipeline-sim/index.html in a browser.

Project Links

GitHub repository: open to view

Live deployment: open to view

Hardware: a personal laptop. Software: Chrome, a text editor, and a browser console for debugging. The DFA transition table, tokeniser, and parser are in one file: js/lexer.js.

Algorithm

Character classification

1. Read one character from the input.
2. Map it to a character class using a classifier function: alpha, digit, sign, hash, comma, lp, rp, colon, ws, nl, eof, or other.

DFA-based tokenisation

1. Start in state START.
2. Look up δ[state][charClass] to get the next state.
3. If the next state continues the current token (e.g. IDENT → IDENT on alpha), advance and repeat.
4. If the next state is a different initial category or ERROR, emit the current token and reset to START.
5. After emitting, classify the token: if the lexeme matches a known opcode, type is OPCODE; if it matches R followed by digits 0–31, type is REGISTER; otherwise LABEL_REF. Immediates and punctuation are classified by the state that produced them.

Parser

1. Walk the token stream left to right.
2. When an OPCODE token is seen, look up its format (R_TYPE, I_TYPE, LOAD, STORE, BRANCH, JUMP, NOP, HLT).
3. Consume the expected sequence of REGISTER, COMMA, IMMEDIATE, LPAREN, RPAREN tokens.
4. Emit a structured instruction object with fields: op, rd, rs1, rs2, imm, label, format.

Source Code

The full lexer is in *pipeline-sim/js/lexer.js*. The fragment below is the core of the DFA transition table and the tokenisation loop.

```
const S = Object.freeze({
  START:0, IDENT:1, REG_R:2, REG_DIG:3,
  IMM_SIGN:4, IMM_DIG:5, HASH:6,
  LPAREN:7, RPAREN:8, COMMA:9, COLON:10,
  COMMENT:11, ERROR:12, WS:13,
});
```

```
function buildDelta() {
  const d = {};
  for (const s of Object.values(S)) {
    d[s] = {};
    for (const cc of ['alpha','digit','sign','hash',
      'comma','lp','rp','colon','ws','nl','eof','other'])
      d[s][cc] = S.ERROR;   // default
  }
  d[S.START]['alpha'] = S.IDENT;
  d[S.START]['digit'] = S.IMM_DIG;
  d[S.START]['sign']  = S.IMM_SIGN;
  d[S.START]['hash']  = S.COMMENT;
  d[S.IDENT]['alpha'] = S.IDENT;
  d[S.IDENT]['digit'] = S.IDENT;
  d[S.IMM_SIGN]['digit'] = S.IMM_DIG;
  d[S.IMM_DIG]['digit']  = S.IMM_DIG;
  // ... remaining transitions
  return d;
}

const DELTA = buildDelta();
```

The transition table is an object indexed by [state][charClass]. Every cell that I did not fill defaults to ERROR, which is the sink state. That is the same convention we use in Module 1 when we say a DFA is complete: every (state, symbol) pair has a defined transition.

Chapter III – Results

Table 1 shows the token output for a sample program. Each token has the type the DFA assigned.

Input line	Tokens produced
ADD R1, R2, R3	OPCODE(ADD) REGISTER(R1) COMMA REGISTER(R2) COMMA REGISTER(R3)
LW R4, 0(R5)	OPCODE(LW) REGISTER(R4) COMMA IMMEDIATE(0) LPAREN REGISTER(R5) RPAREN
loop: BEQ R1, R2, done	LABEL_DEF(loop) OPCODE(BEQ) REGISTER(R1) COMMA REGISTER(R2) COMMA LABEL_REF(done)
ADDI R1, R0, -5	OPCODE(ADDI) REGISTER(R1) COMMA REGISTER(R0) COMMA IMMEDIATE(-5)
# comment	(no tokens — comment consumed by COMMENT state)
HLT	OPCODE(HLT)

Table 2 is a partial dump of the DFA transition table as rendered in the application. Each row is a state; each column is a character class. The table is generated from the same DELTA object the tokeniser uses.

State	alpha	digit	sign	hash	comma	ws	other
START	IDENT	IMM_DIG	IMM_SIGN	COMMENT	COMMA	WS	ERROR
IDENT	IDENT	IDENT	ERROR	ERROR	ERROR	ERROR	ERROR
IMM_SIGN	ERROR	IMM_DIG	ERROR	ERROR	ERROR	ERROR	ERROR
IMM_DIG	ERROR	IMM_DIG	ERROR	ERROR	ERROR	ERROR	ERROR
COMMENT	COMMENT	COMMENT	COMMENT	COMMENT	COMMENT	COMMENT	COMMENT

The DFA has 14 states and 12 character classes, giving a 14 × 12 = 168 cell table. Most cells are ERROR. That is normal: a real scanner's DFA is sparse. The tokeniser processes a 10-line program in under 1 ms. Correctness was checked by comparing the token output against hand-written expected lists for 8 test programs covering all instruction formats, labels, comments, and negative immediates. All matched.

On the screen, each token type is shown with a different colour: opcodes are blue, registers are purple, immediates are amber, labels are green. This colour coding comes directly from the DFA output; the UI just maps token types to CSS classes. If the DFA assigned the wrong type, the wrong colour would appear, so the visual display is itself a quick sanity check.

Chapter IV – Conclusion

The problem was to turn raw assembly text into typed tokens using a DFA, not a library regex. The lexer does that. It has an explicit transition table with 14 states and 12 character classes. Every character is classified once, looked up in the table once, and the state advances. The result is a token stream that the parser and pipeline engine can trust.

What I understood from this part of the project is that a DFA is not just a diagram for exam problems. It is a real data structure. The transition table is a 2D array, the current state is an integer, and the scan loop is a while loop with one table lookup per iteration. That is O(n) with a small constant, which is why real compilers use exactly this approach (Aho et al., 2006).

The limit of this lexer is that it does not handle string literals, floating point, or multi-line constructs. Those would need more states or a different approach (for floating point, a second DFA or a state with a dot transition). But for the RISC assembly set I defined, the current DFA is complete and correct.

The FLAT concept — deterministic finite automata and tokenisation from Modules 1 and 2 — is clearly implemented and visible in the application. The user can inspect the DFA table, see the tokens it produces, and trace any character through the transition function.

References

1. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson.
2. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Addison-Wesley. (Chapter 3: Lexical Analysis.)
3. Thompson, K. (1968). Programming techniques: Regular expression search algorithm. *Communications of the ACM, 11*(6), 419–422.